Search for Movie

```
<html>
<head>
        <title>Nderon Beqiri</title>
        <LINK REL='stylesheet' TYPE='text/css' HREF='dbicw.css'>
</head>
<body>
<h1>Movie Search Result</h1>

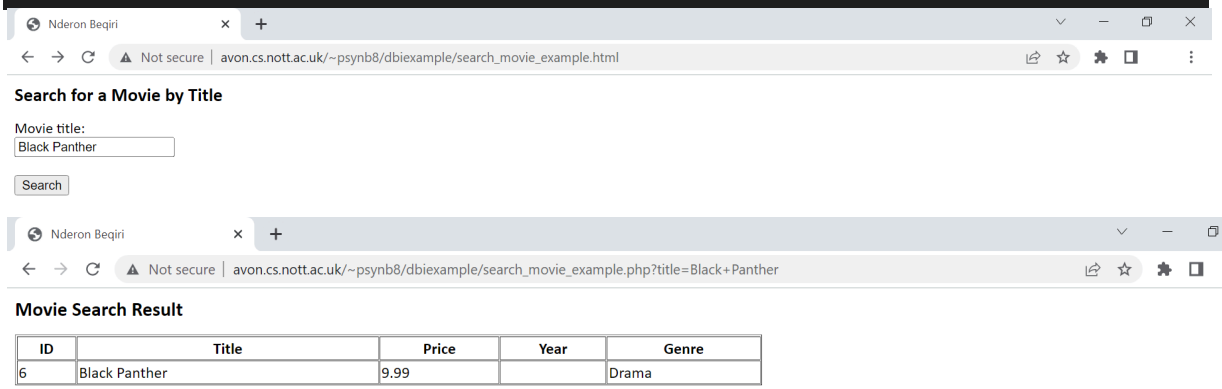
<?php
$titlesearch = $_GET['title'];
$db_host = 'mysql.cs.nott.ac.uk';
$db_user = 'psynb8_COMP1004';
$db_pass = 'rV_r516';
$db_name = 'psynb8_COMP1004';

$conn = new mysqli($db_host, $db_user, $db_pass, $db_name);
if ($conn->connect_errno)  die("failed to connect to database\n</body>\n</html>");

$sql="SELECT mvID,title,price,year,genre FROM Movie WHERE title='$titlesearch'";
$stmt = $conn->prepare($sql);
$stmt->execute();
$stmt->bind_result($ID, $Title, $Price, $Year, $Genre );
?>

<table width="750" border="1" cellpadding="1" cellspacing="1">
<tr> <th>ID</th> <th>Title</th> <th>Price</th> <th>Year</th> <th>Genre</th> </tr>

<?php
while($stmt->fetch()){
  echo "<tr>";
  echo "<td>". htmlentities($ID) ."</td>";
  echo "<td>". htmlentities($Title) ."</td>";
  echo "<td>". htmlentities($Price) ."</td>";
  echo "<td>". htmlentities($Year) ."</td>";
  echo "<td>". htmlentities($Genre) ."</td>";
  echo "</tr>";
}
?>
</table>
</body>
</html>
```



ID	Title	Price	Year	Genre
6	Black Panther	9.99		Drama

Here we use PHP and JS to establish a connection to the server. We then execute SQL commands within that server to manipulate the interface of the website. The <tr> (row), <th> (header), <td> (data cell), commands within the <table> tags are used to create a table and each data cell is assigned to individual columns of our database of which I binded to different variables.

Search for Actor

```
<html>
<head>
        <title>Nderon Beqiri</title>
        <LINK REL='stylesheet' TYPE='text/css' HREF='dbicw.css'>
</head>
<body>
<h1>Actor Search Result</h1>

<?php
$titlesearch = $_GET['actName'];
$db_host = 'mysql.cs.nott.ac.uk';
$db_user = 'psynb8_COMP1004';
$db_pass = 'rV_r516';
$db_name = 'psynb8_COMP1004';

$conn = new mysqli($db_host, $db_user, $db_pass, $db_name);
if ($conn->connect_errno)  die("failed to connect to
database\n</body>\n</html>");

$sql="SELECT actID,actName FROM Actor WHERE actName='$titlesearch'";
$stmt = $conn->prepare($sql);
$stmt->execute();
$stmt->bind_result($actID, $actName);
?>

<table width="750" border="1" cellpadding="1" cellspacing="1">
<tr> <th>actID</th> <th>actName</th> </tr>

<?php
while($stmt->fetch()){
  echo "<tr>";
  echo "<td>". htmlentities($actID) ."</td>";
  echo "<td>". htmlentities($actName) ."</td>";
  echo "</tr>";
}
?>
</table>
</body>
</html>
```

Here, we simply make tweaks to the previous code in a new file where we change the table SQL SELECTS FROM and the attributes it selects. Also the variable $titlesearch is changed so it can match what is found in our Actor table. The variables are now also binded to the columns in the Actor table, actID and actName.

```
<html>
<head>
        <title>Nderon Beqiri</title>
        <LINK REL='stylesheet' TYPE='text/css' HREF='dbicw.css'>
        <script type="text/javascript" src="dbicw.js"></script>

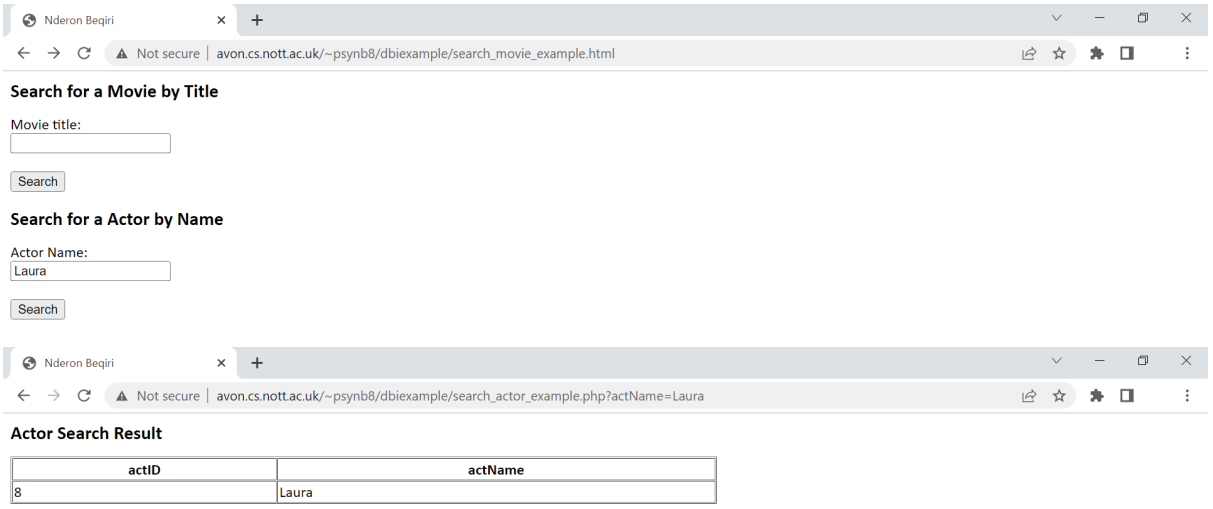
</head>
<body>

<h1>Search for a Movie by Title</h1>

<form action="search_movie_example.php"  onSubmit="return validate(this)">
Movie title:<br>
 <input type="text" name="title">
 <br>
 <br>
 <input type="submit" value="Search">
</form>

<h1>Search for a Actor by Name</h1>
<form action="search_actor_example.php"  onSubmit="return validate(this)">
Actor Name:<br>
 <input type="text" name="actName">
 <br>
 <br>
 <input type="submit" value="Search">
</form>

</body>
</html>
```



Here we create the textboxes which take in a keyboard input and require the click of a Search button to proceed. Notice how <form action=...> is different for both files as their names are different and we must link the correct things to produce the right web result.

Deleting Movie from Database

```
<html>
<head>
        <title>Nderon Beqiri</title>
        <LINK REL='stylesheet' TYPE='text/css' HREF='dbicw.css'>
</head>
<body>
<h1>Movie Deleted Result</h1>

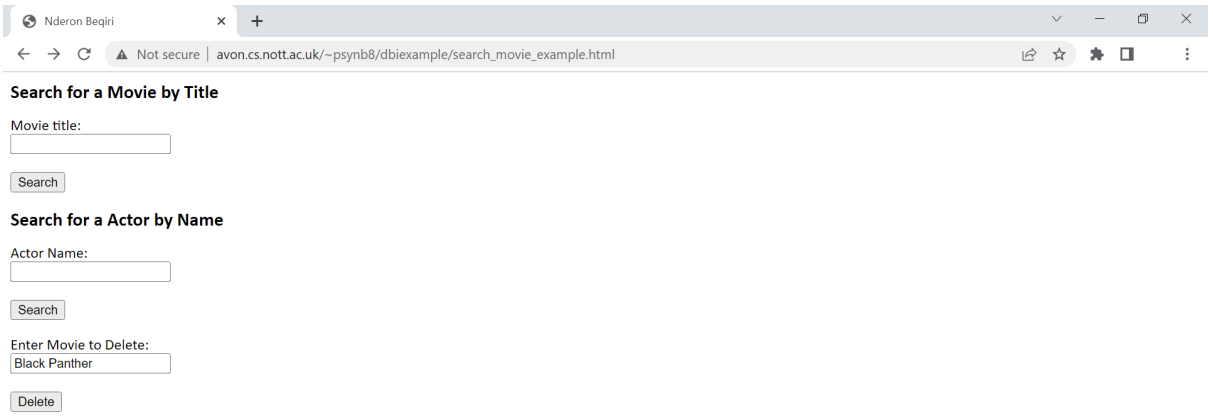
<?php
$titlesearch = $_GET['title'];
$db_host = 'mysql.cs.nott.ac.uk';
$db_user = 'psynb8_COMP1004';
$db_pass = 'rV_r516';
$db_name = 'psynb8_COMP1004';

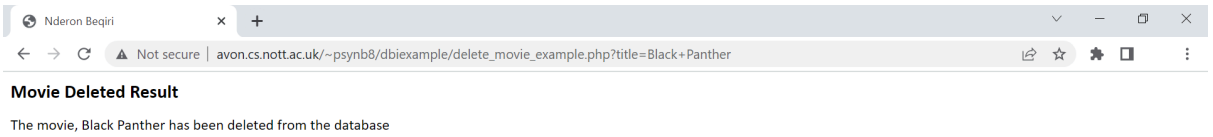
$conn = new mysqli($db_host, $db_user, $db_pass, $db_name);
if ($conn->connect_errno)  die("failed to connect to
database\n</body>\n</html>");

$sql="DELETE FROM Movie WHERE title='$titlesearch'";
$stmt = $conn->prepare($sql);
$stmt->execute();
$stmt->bind_result($ID, $Title, $Price, $Year, $Genre );
echo "The movie, $titlesearch has been deleted from the database"
?>
</table>
</body>
</html>
```

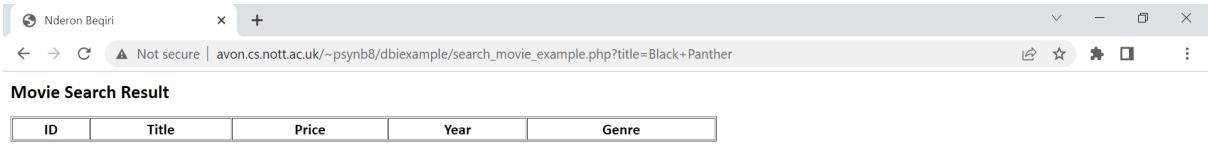
```
//within body and html tags
<form action="delete_movie_example.php"  onSubmit="return
validate(this)">
        Enter Movie to Delete:<br>
         <input type="text" name="title">
         <br>
         <br>
         <input type="submit" value="Delete">
        </form>
```

1- Name of movie to delete is entered
2- The database removes it and shows what movie has been removed
3- You search to see if the movie you wanted to delete is still there
4- The movie is no longer there and has been successfully deleted









The code here is a minor tweak to the Search Movie code. Simply, the SQL is changed to DELETE a row where the user enters into the variable $titlesearch if it matches the name of a movie in the database. Furthermore, the PHP keyword echo, prints to the screen informing the user what movie they have removed from the database.
I have also changed the button clicked to Delete, rather than Search so it is clear to the user the function of the button, and the form action to the name of the delete movie file.

Deleting Actor from Database

```
<html>
<head>
        <title>Nderon Beqiri</title>
        <LINK REL='stylesheet' TYPE='text/css' HREF='dbicw.css'>
</head>
<body>
<h1>Actor Deleted Result</h1>

<?php
$titlesearch = $_GET['actID'];
$titlesearch1 = $_GET['actID'];
$db_host = 'mysql.cs.nott.ac.uk';
$db_user = 'psynb8_COMP1004';
$db_pass = 'rV_r516';
$db_name = 'psynb8_COMP1004';

$conn = new mysqli($db_host, $db_user, $db_pass, $db_name);
if ($conn->connect_errno)  die("failed to connect to
database\n</body>\n</html>");

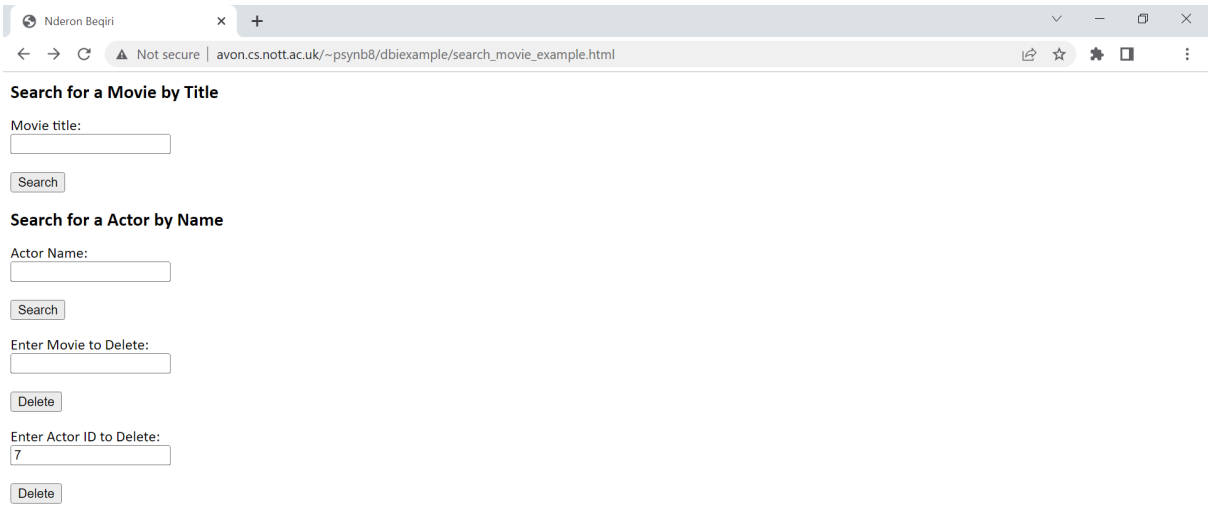
$sql="DELETE FROM Movie WHERE actID='$titlesearch'";
$stmt = $conn->prepare($sql);
$stmt->execute();
$stmt->bind_result($ID, $Title, $Price, $Year, $Genre );

$sql1="DELETE FROM Actor WHERE actID='$titlesearch'";
$stmt1 = $conn->prepare($sql1);
$stmt1->execute();
$stmt1->bind_result($actID,$actName);
echo "The actor, with actor ID $titlesearch has been deleted from the database"
?>
</table>
</body>
</html>
```

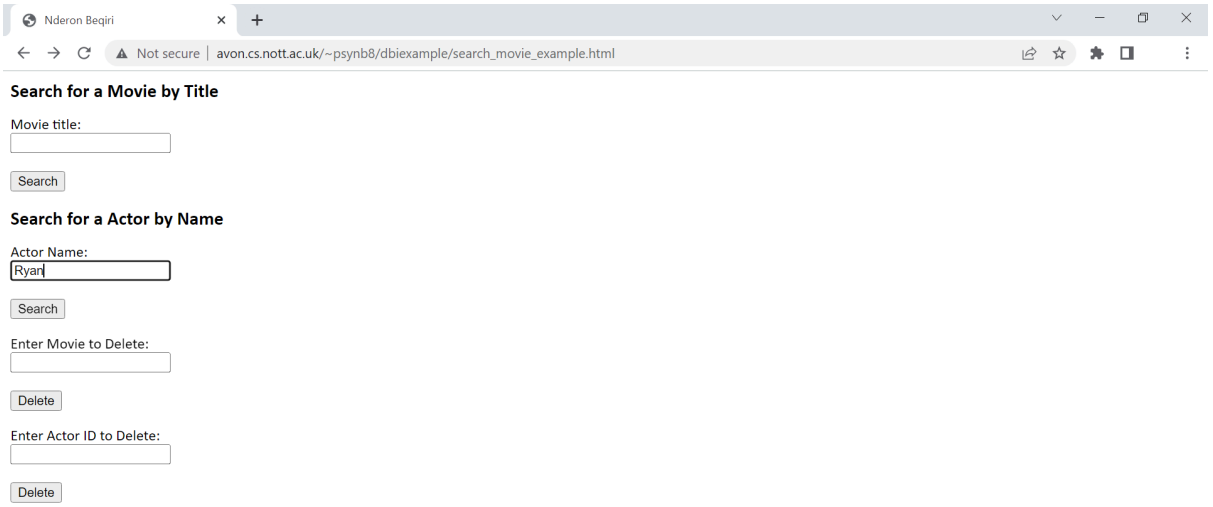
```
//within html and body tags
<form action="delete_actor_example.php"  onSubmit="return validate(this)">
Enter Actor ID to Delete:<br>
 <input type="text" name="actID">
 <br>
 <br>
```

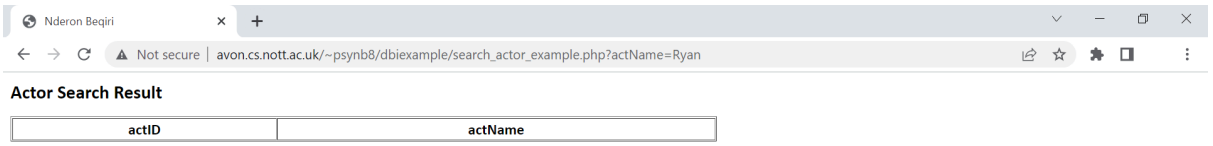
```
 <input type="submit" value="Delete">
</form>
```

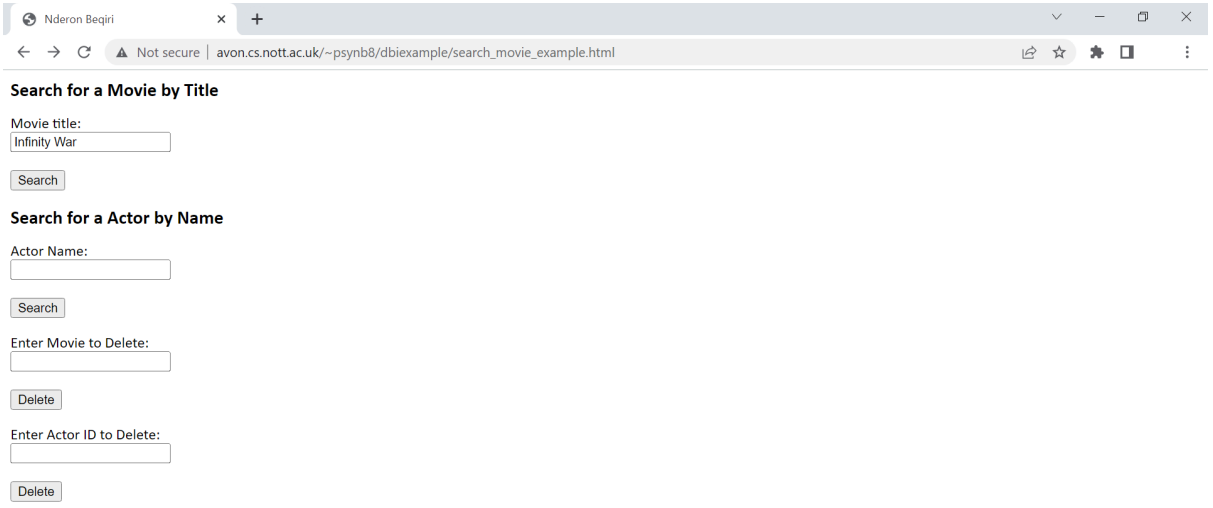
1- Enter the actID of which actor to delete as it is a foreign key in Movie table
2- It verifies the ID of the actor you deleted
3- Enter the actors name of whom you previously deleted when entering ID
4- Their record will be deleted from the Actor table
5- Enter the movie the deleted actor starred in
6- You will see that record has been deleted

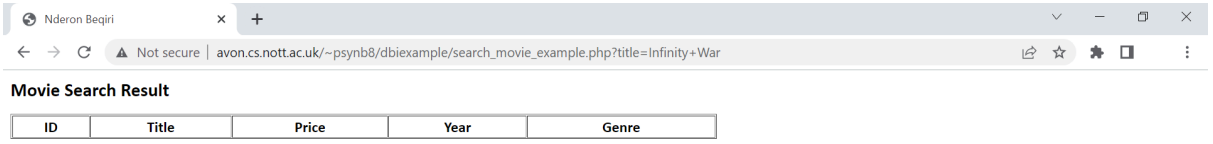












The SQL here is much more complex in contrast to removing a movie in the database. As we are removing an Actor, and each actor has an individual actID which is a foreign key to the Movie table, we first must DELETE a row FROM Movie WHERE the actID is matched to the user's input. Now there is no foreign key for that individual actor, which is more useful than DROPping the whole actID column in the Movie table. As a result, we can now DELETE a row FROM Actor WHERE the actID matches the one previously entered and remove a record of the actor.

Inserting Actor into Database

```
<html>
<head>
        <title>Nderon Beqiri</title>
        <LINK REL='stylesheet' TYPE='text/css' HREF='dbicw.css'>
</head>
<body>
<h1>Actor Search Result</h1>

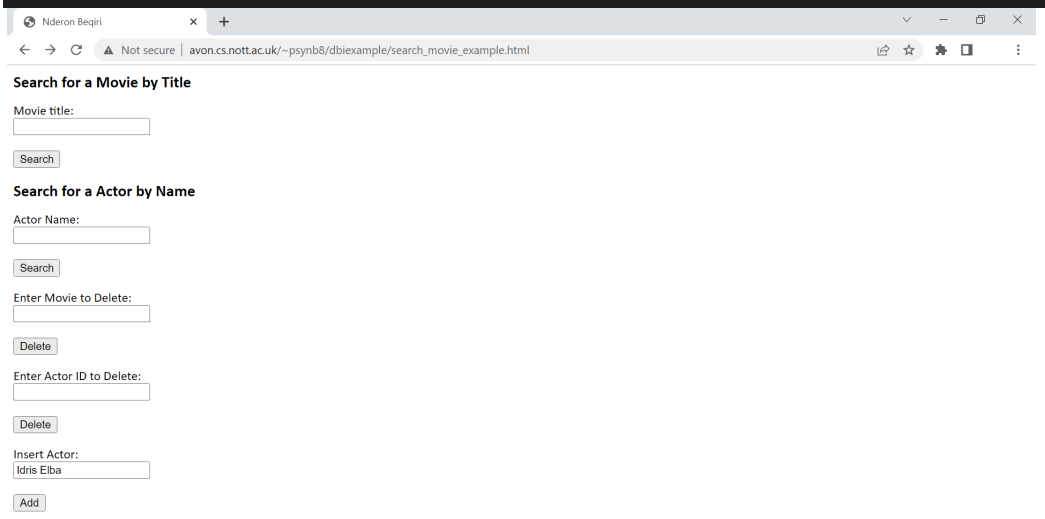
<?php
$actorName = $_GET['actorAdd'];
$db_host = 'mysql.cs.nott.ac.uk';
$db_user = 'psynb8_COMP1004';
$db_pass = 'rV_r516';
$db_name = 'psynb8_COMP1004';

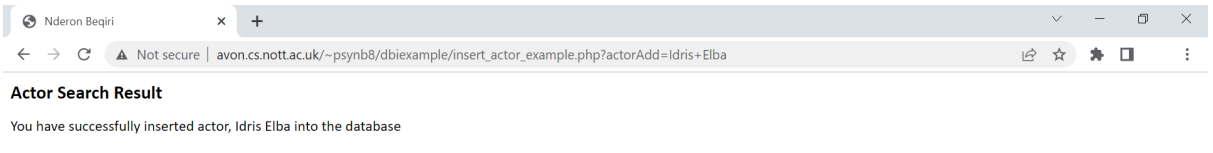
$conn = new mysqli($db_host, $db_user, $db_pass, $db_name);
if ($conn->connect_errno)  die("failed to connect to database\n</body>\n</html>");

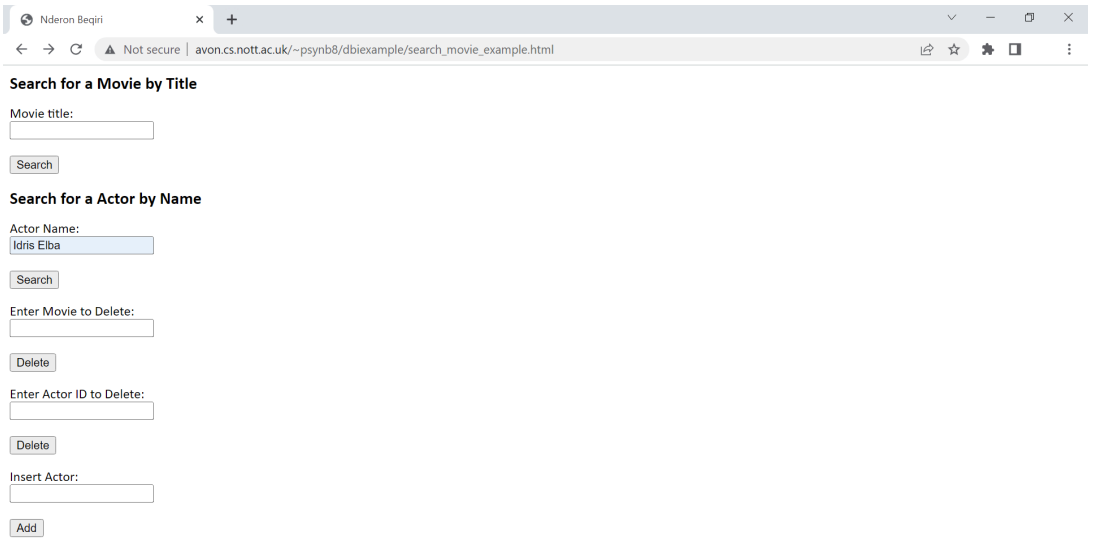
$sql="INSERT INTO Actor(actName) VALUES ('$actorName')";
$stmt = $conn->prepare($sql);
$stmt->execute();
$stmt->bind_result($actID, $actName);
echo "You have successfully inserted actor, $actorName into the database"
?>

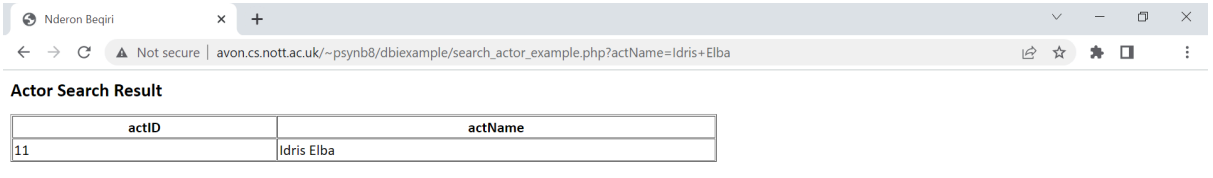
</body>
</html>
```

```
//within html and body tags
<form action="insert_actor_example.php"  onSubmit="return validate(this)">
        Insert Actor:<br>
         <input type="text" name="actorAdd">
         <br>
         <br>
         <input type="submit" value="Add">
        </form>
```









actID	actName
11	Idris Elba

Here, the SQL is changed so that we INSERT specified VALUES, in this case only the name of the actor, into the Actor table. Notice how we do not insert the actID manually; this is because it automatically indents for each new actor inserted into the table.
We also changed the submit button to Add to make the functionality of the button clearer.

Inserting Movie into Database

```
<html>
<head>
        <title>Nderon Beqiri</title>
        <LINK REL='stylesheet' TYPE='text/css' HREF='dbicw.css'>
</head>
<body>
<h1></h1>

<?php
$title = $_GET['movieAdd'];
$genre = $_GET['genreAdd'];
$price = $_GET['priceAdd'];
$actId = $_GET['actorIDAdd'];

$db_host = 'mysql.cs.nott.ac.uk';
$db_user = 'psynb8_COMP1004';
$db_pass = 'rV_r516';
$db_name = 'psynb8_COMP1004';

$conn = new mysqli($db_host, $db_user, $db_pass, $db_name);
if ($conn->connect_errno)  die("failed to connect to database\n</body>\n</html>");

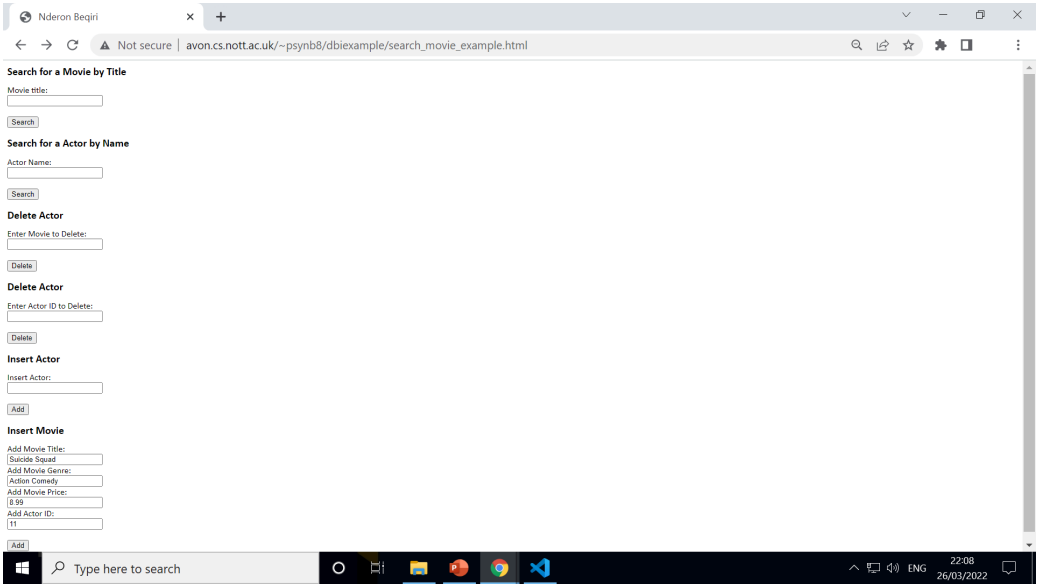
$sql="INSERT INTO Movie(actID,price,title,genre) VALUES ($actId,$price,'$title','$genre')";
$stmt = $conn->prepare($sql);
$stmt->execute();
$stmt->bind_result($mvID,$actId,$year,$price, $title, $genre );
echo "You have successfully added $title to your Movie Database";
?>

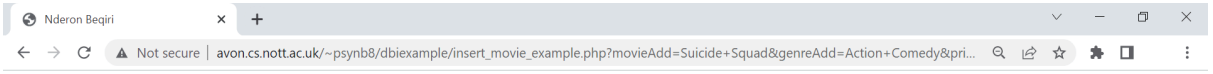

</table>
</body>
</html>
```

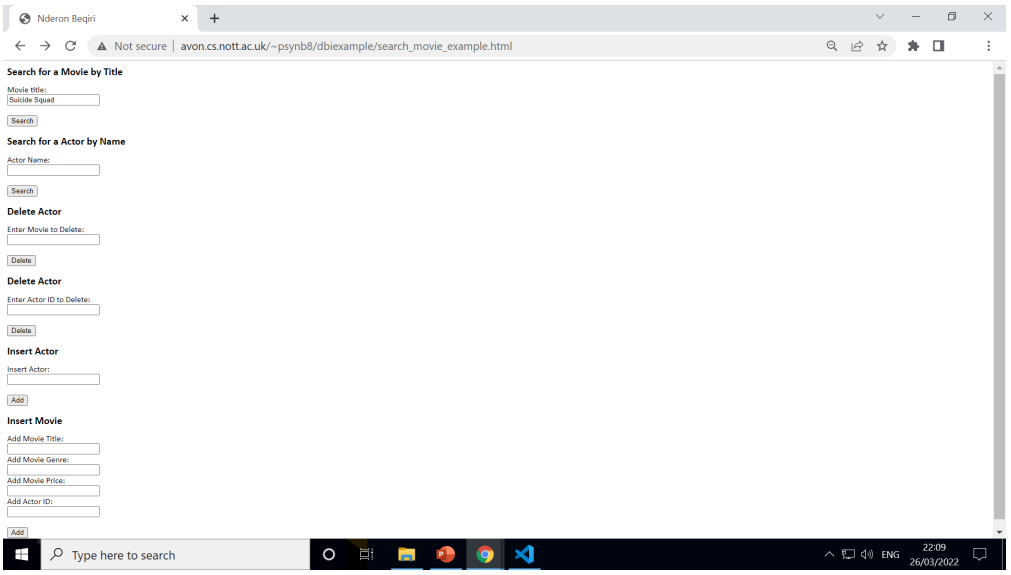
```
//within html and body tags
<h1>Insert Movie</h1>
        <form action="insert_movie_example.php"  onSubmit="return validate(this)">
                Add Movie Title:<br>
                 <input type="text" name="movieAdd">
                 <br>
                 Add Movie Genre:<br>
                 <input type="text" name="genreAdd">
                 <br>
                 Add Movie Price:<br>
                 <input type="text" name="priceAdd">
                 <br>

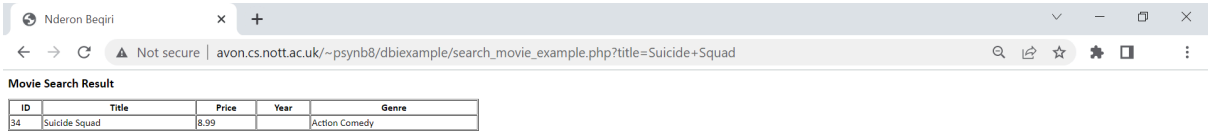
                 Add Actor ID:<br>
                 <input type="text" name="actorIDAdd">
                 <br>

                 <br>
                 <input type="submit" value="Add">
                </form>
```









Similarly to adding actors to the database, we use INSERT and the designated VALUES again. This time however we add these values to the movie table and there are 4 values to add rather than just the one. The $title and $genre variables are within quotations as they are strings, contrastingly to the $price and $year. As with adding actor to the database where we did not enter actID, the same applies here with mvID as it automatically indents after each insertion.
However the format of submission is different with 4 boxes, one for each value being submitted with a clear message above the box stating which value to enter. These will all be processed at the same time and will be inserted into the database upon clicking the Add button at the bottom of the page.